

IA Embarcada e Modelos Compactos · Projeto final

# Reconhecimento de atividade humana no ESP32-S3

Um acelerômetro e um giroscópio, uma rede neural de 4630 parâmetros comprimida para int8, e tudo rodando no chip, simulado no Wokwi.



# O problema

Dado o movimento de uma pessoa, dizer o que ela está fazendo: andando, subindo escada, descendo escada, sentada, em pé ou deitada.

O modelo roda dentro de um microcontrolador, perto do sensor. Assim a decisão acontece no próprio dispositivo, sem mandar o sinal bruto para outro lugar.

Isso obriga o modelo a ser pequeno. Neste projeto ele ficou com 9856 bytes.

## O que o enunciado pede

- coleta de dados de sensores
- treino com dataset público
- conversão e compressão do modelo
- pipeline de inferência no dispositivo

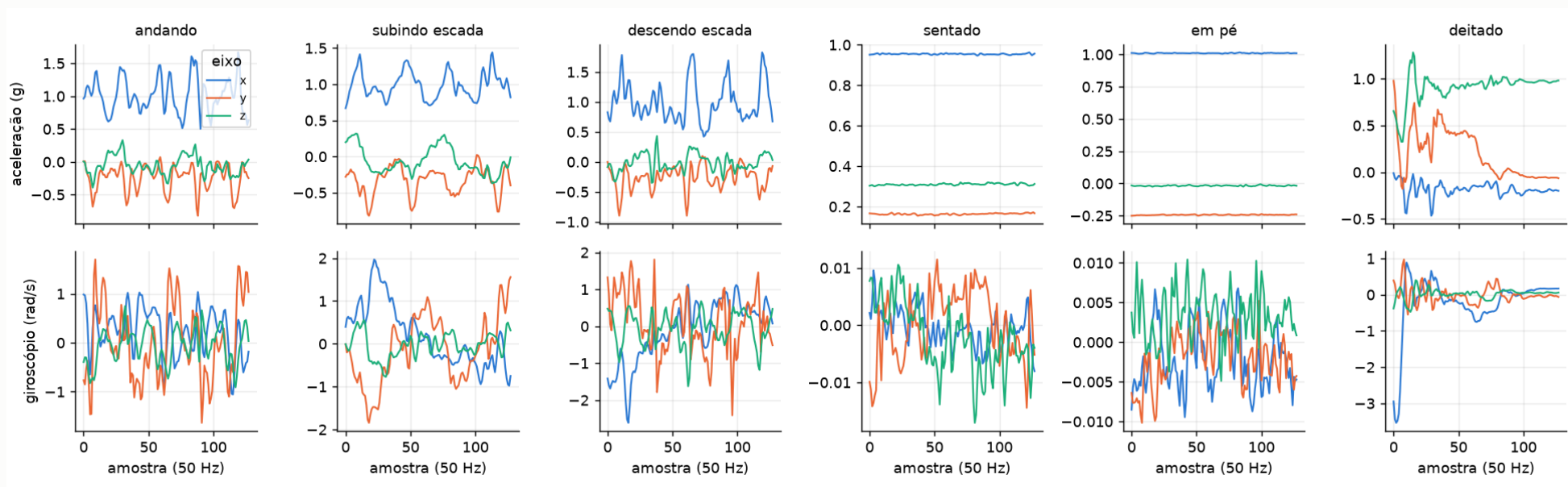
Hardware: ESP32-S3 e MPU6050 (acelerômetro e giroscópio), simulados no Wokwi.

# Os dados: UCI HAR

30 voluntários com um celular na cintura, 6 atividades, sensor a 50 Hz. Os dados já vêm em janelas de 128 amostras (2,56 s).

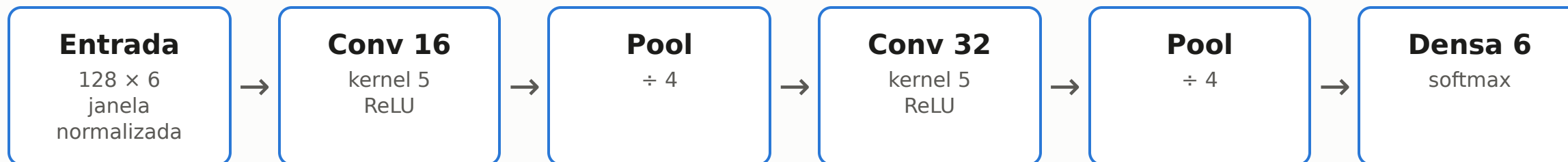
São usados 6 canais, os mesmos que o MPU6050 entrega: aceleração (x, y, z, em g) e giroscópio (x, y, z, em rad/s).

| CONJUNTO  | JANELAS | PESSOAS |
|-----------|---------|---------|
| Treino    | 5952    | 17      |
| Validação | 1400    | 4       |
| Teste     | 2947    | 9       |



Uma janela de cada atividade. A separação é por pessoa: o teste mede pessoas que o modelo nunca viu.

# O modelo



Cada convolução desliza pelo tempo procurando padrões, como o balanço de uma passada. O pooling reduz o tamanho. A camada final dá a probabilidade de cada atividade.

O Conv2D com kernel (5, 1) é, na prática, uma convolução 1D. Foi escolhido por ser a convolução mais bem suportada no TFLite Micro.

**4630**

parâmetros

**5**

operações do TFLite no modelo final

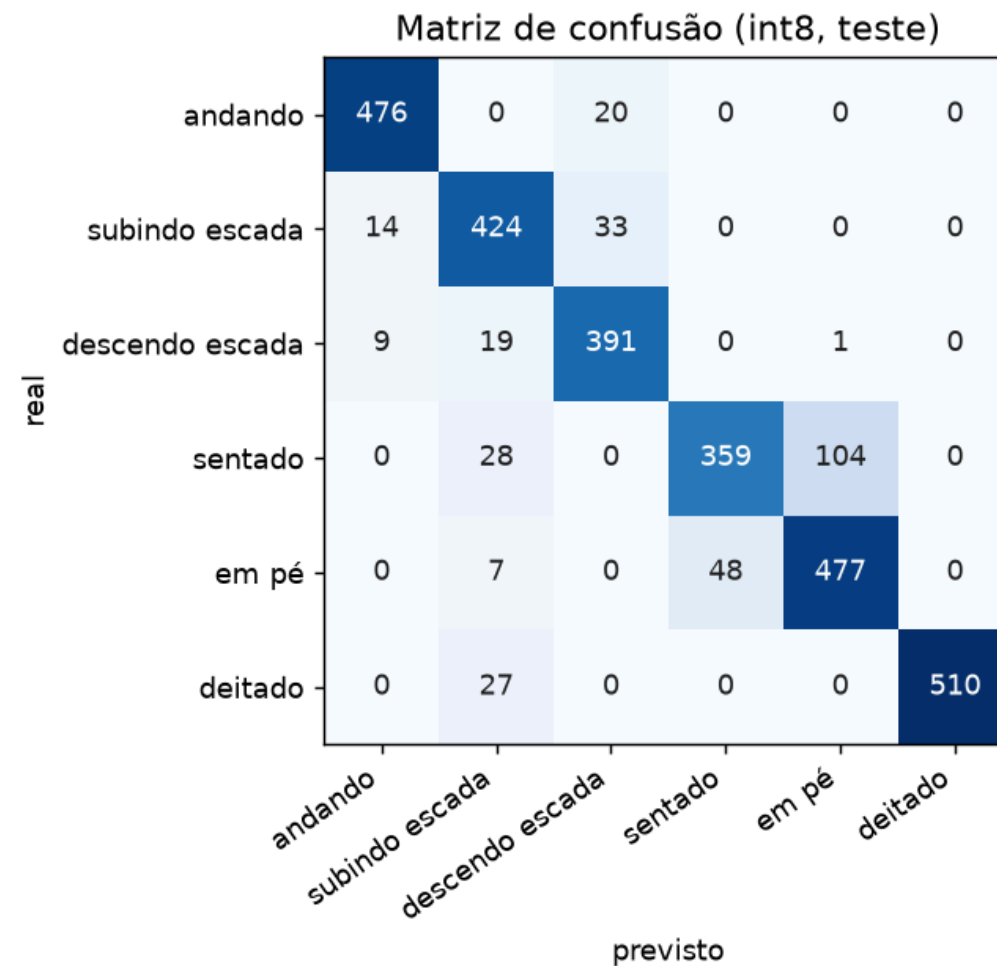
| CAMADA                         | PARÂMETROS |
|--------------------------------|------------|
| Conv 16 (6 canais, kernel 5)   | 496        |
| Conv 32 (16 canais, kernel 5)  | 2592       |
| Densa (256 entradas, 6 saídas) | 1542       |

# Resultado do treino

## 89,5%

acurácia em 2947 janelas de 9 pessoas novas  
(modelo int8)

| ATIVIDADE       | ACERTO |
|-----------------|--------|
| andando         | 96,0%  |
| subindo escada  | 90,0%  |
| descendo escada | 93,1%  |
| sentado         | 73,1%  |
| em pé           | 89,7%  |
| deitado         | 95,0%  |



O maior erro é sentado previsto como em pé (104 de 491). São duas posturas paradas e o sensor sente quase a mesma coisa.

# Compressão para int8

Os números de float32 (4 bytes) são trocados por inteiros de 8 bits, com uma escala e um ponto zero para cada tensor. A escala vem de 300 janelas de treino de exemplo.

|                      |                                                                                    |                        |
|----------------------|------------------------------------------------------------------------------------|------------------------|
| float32              |  | 21860 B · 89,45%       |
| faixa dinâmica       |   | 10120 B · 89,41%       |
| <b>int8 completo</b> |   | <b>9856 B · 89,48%</b> |

**2,2x**

menor que o float32

**1 janela**

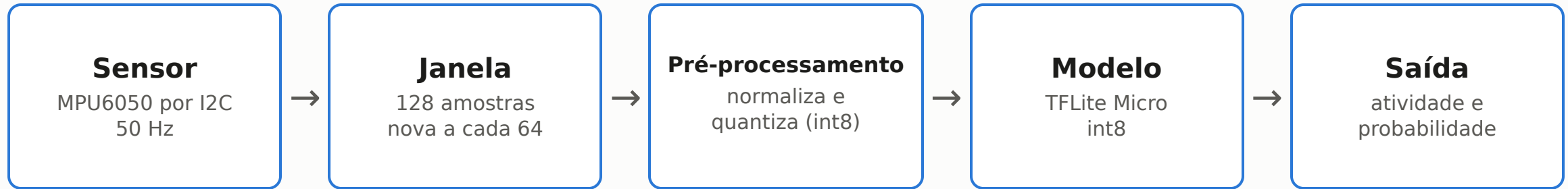
de diferença na acurácia entre float e int8 (em 2947)

**99,66%**

das janelas têm a mesma resposta em float e int8

O chip faz as contas em inteiros, que são mais rápidas e leves. O arquivo não cai exatamente 4x, porque guarda também a estrutura do modelo e os parâmetros de quantização.

# O pipeline dentro do chip



A inferência leva centenas de milissegundos e as amostras chegam a cada 20 ms. Por isso a leitura do sensor fica numa tarefa do FreeRTOS com prioridade maior, e a inferência na tarefa principal.

Assim nenhuma amostra se perde enquanto o modelo roda: a tarefa do sensor monta a janela e avisa a tarefa principal quando ela está completa.

# Demonstração no Wokwi

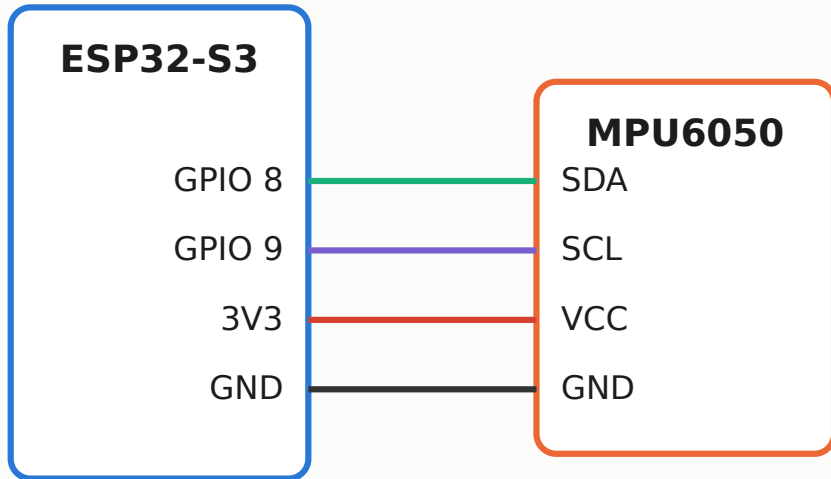


diagrama da ligação (diagram.json)

O MPU6050 do Wokwi tem controles de aceleração e giro que ficam parados. Para ele se mexer, um cenário do `wokwi-cli` muda os controles a cada 20 ms com uma janela real do dataset.

```
# saída serial do chip simulado, cenário "andando" (log completo em simulacao/logs/)
# | real          | previsto          | prob | pre(us) | inf(us) | igual ao PC
0 | andando        | andando           | 1.00 | 11019   | 279230   | sim
...
36 | sentado         | em pe             | 0.67 | 10700   | 279299   | sim
...
Acuracia no chip: 58/60 (96.7%)
Saída int8 identica a do PC: 60/60 janelas
Tempo medio: pre-processamento 10841 us, inferencia 279276 us

=== MOD0 LIVE: lendo o MPU6050 a 50 Hz ===
[live] janela 1: andando          (prob 1.00)
```

# Verificação no chip simulado

60/60

janelas com saída int8 idêntica à do PC

58/60

janelas classificadas certo no replay

6/6

atividades certas no modo live, lendo o MPU6050 simulado

| ATIVIDADE REAL  | PREVISTO PELO CHIP (LIVE) | PROB. |
|-----------------|---------------------------|-------|
| andando         | andando                   | 1,00  |
| subindo escada  | subindo escada            | 1,00  |
| descendo escada | descendo escada           | 0,89  |
| sentado         | sentado                   | 0,76  |
| em pé           | em pé                     | 0,98  |
| deitado         | deitado                   | 1,00  |

As 128 amostras lidas pelo firmware bateram com o dataset (diferença máxima de 0,0003). Resultados iguais no ESP-IDF 5.5.5 e 6.1. O modo live testou uma janela por atividade: mostra que a cadeia funciona, não mede acurácia.

# Limitações e conclusão

## Limitações

- Sentado e em pé se confundem (73% de acerto em sentado).
- O dataset vem de um celular na cintura. Outra posição pede dados novos.

## O que ficou

- 4630 parâmetros, 9856 bytes, 89,5% em pessoas novas.
- int8 deixou o modelo 2,2× menor sem perder acurácia.
- O chip simulado dá a mesma saída que o PC e reconhece as 6 atividades lendo o sensor.
- Próximo passo: testar em hardware e melhorar sentado × em pé.

Dataset: Anguita et al., 2013 (UCI HAR, CC BY 4.0). TensorFlow Lite Micro via espressif/esp-tflite-micro.